
Terraform Notes – Count **VS** For_Each

Purpose

Both `count` and `for_each` help create **multiple resources** from a single block in Terraform.

COUNT

Definition:

Creates multiple copies of a resource using a **number**.

Shell

```
resource "aws_instance" "web" {  
  ami           = var.ami  
  instance_type = var.instance_type  
  count         = 3  
}
```

State references:

Shell

```
aws_instance.web[0]  
aws_instance.web[1]  
aws_instance.web[2]
```

Dynamic Count Using a List

Shell

```
variable "webservers" {  
  type    = list(string)  
  default = ["web1", "web2", "web3"]  
}  
  
resource "aws_instance" "web" {  
  count = length(var.webservers)  
  tags = {  
    Name = var.webservers[count.index]  
  }  
}
```

- ✅ Creates instance per list element
- ✅ Easy scaling by changing list length

⚠️ Count Limitation

- Works by **index**, not by name.
- If you **remove or reorder** list items → Terraform may update or destroy wrong resources.

🧠 Example:

If you remove "web1", then:

Shell

```
web2 → replaces web1  
web3 → replaces web2  
web3 (last) → destroyed
```

💡 **Tip:** Always check `terraform plan` before applying.

FOR_EACH

Definition:

Creates resources for each **key** in a **map** or **set**.
Resources are tracked by key — not index.

```
Shell
variable "webservers" {
  type    = set(string)
  default = ["web1", "web2", "web3"]
}

resource "aws_instance" "web" {
  for_each = var.webservers
  tags = {
    Name = each.value
  }
}
```

State references:

```
Shell

aws_instance.web["web1"]

aws_instance.web["web2"]

aws_instance.web["web3"]
```

- ✓ Removing "web1" only removes that instance.
 - ✓ No issue if order changes.
 - ✓ More predictable than `count`.
-

Key Differences

Key Differences

Feature	<code>count</code>	<code>for_each</code>
Input	list / number	set / map
Identifier	Index (<code>count.index</code>)	Key (<code>each.key</code> , <code>each.value</code>)
Reordering impact	Can recreate resources	Stable, safe
Use case	Simple repeats	Named, trackable resources

Remember:

`count` → index-based (fragile)

`for_each` → key-based (stable)